



프로그램 언어

11주차



인하대학교



객체(object)

파이썬에서는 모든 것이 객체이다.

- 파이썬에서는 모든 것이 객체로 구현된다. 정수도 객체이고 문자열도 객체이며 리스트도 객체이다.

```
>>>a=100
>>>type(a)
<class 'int'>
>>>b=[1,2,3,4]
>>>type(b)
<class 'list'>
```

```
>>> "Everything in Python is an object".upper()
'EVERYTHING IN PYTHON IS AN OBJECT'
>>>a_list=['apple','carrot','banana']
>>>a_list.append('kiwi')
>>>a_list.sort()
```

문자열 요소를 속성으로 가질 수 있고(속성)
다양한 기능을 수행하도록 정의(메소드)

파이썬 기본 클래스

list 클래스

속성:

메소드:

str 클래스

속성: utc

메소드:

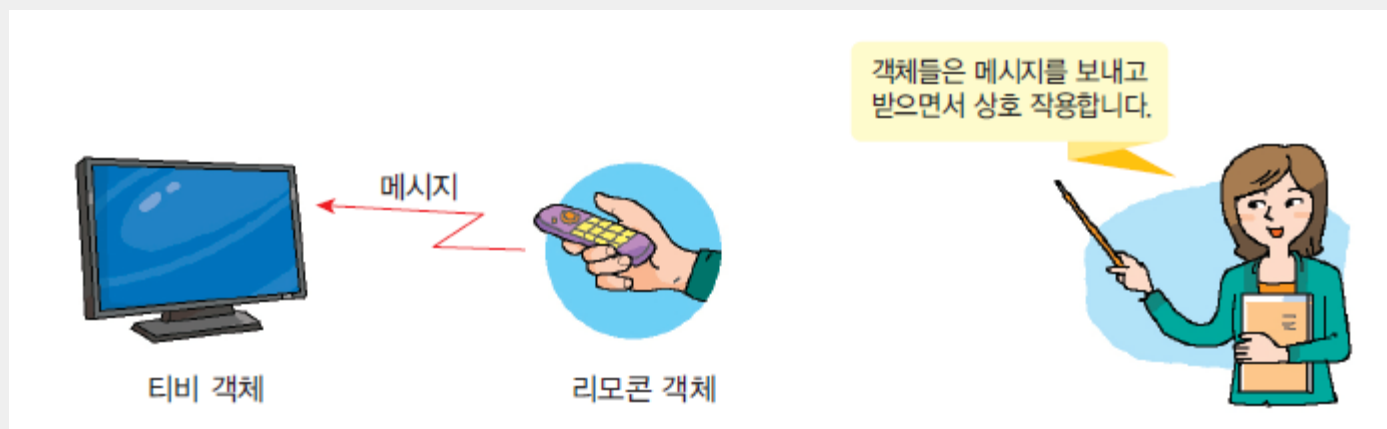
int 클래스

속성:

메소드:

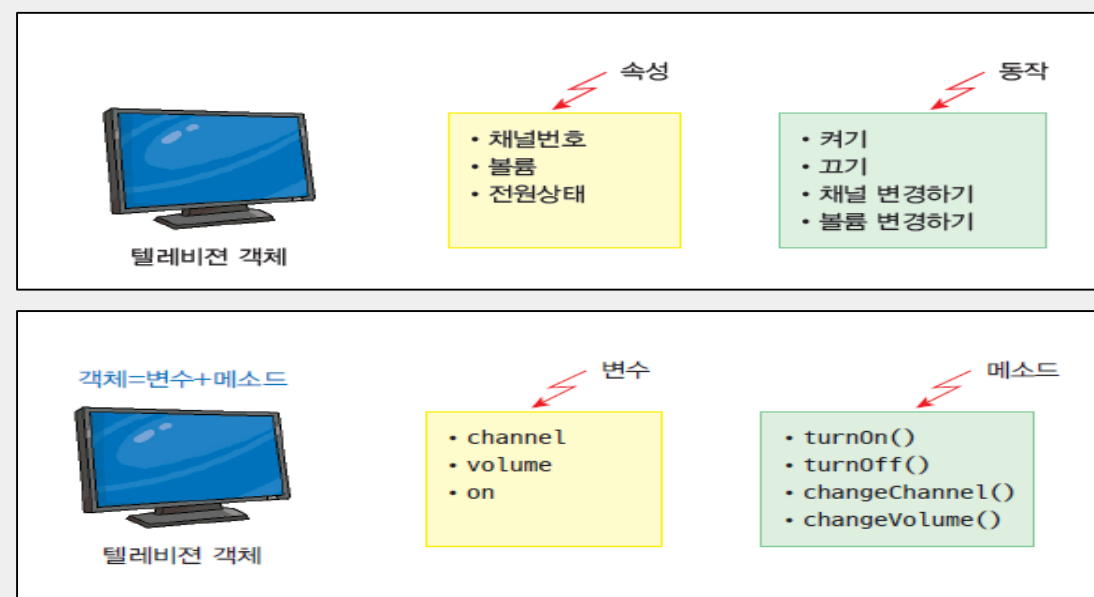
객체 지향 프로그래밍(Object-Oriented Programming)

- 객체를 우선으로 생각해서 프로그래밍하는 것
- 다양한 객체를 미리 정의해둔 클래스에 따라 생성하고, 이들이 프로그램 상에서 상호작용하며 목표에 도달하는 문제해결방식
- 객체 지향 프로그래밍에서는 서로 관련 있는 데이터와 함수를 묶어서 객체(object)로 만들고 이들 객체들이 모여서 하나의 프로그램이 된다.
- 객체를 효율적으로 만들고 사용하는 것이 목표



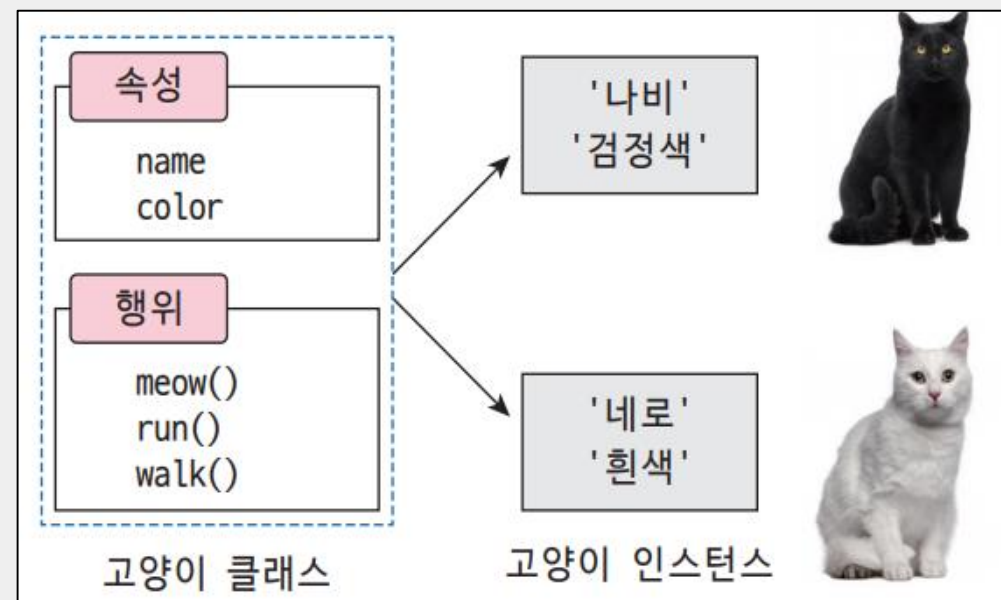
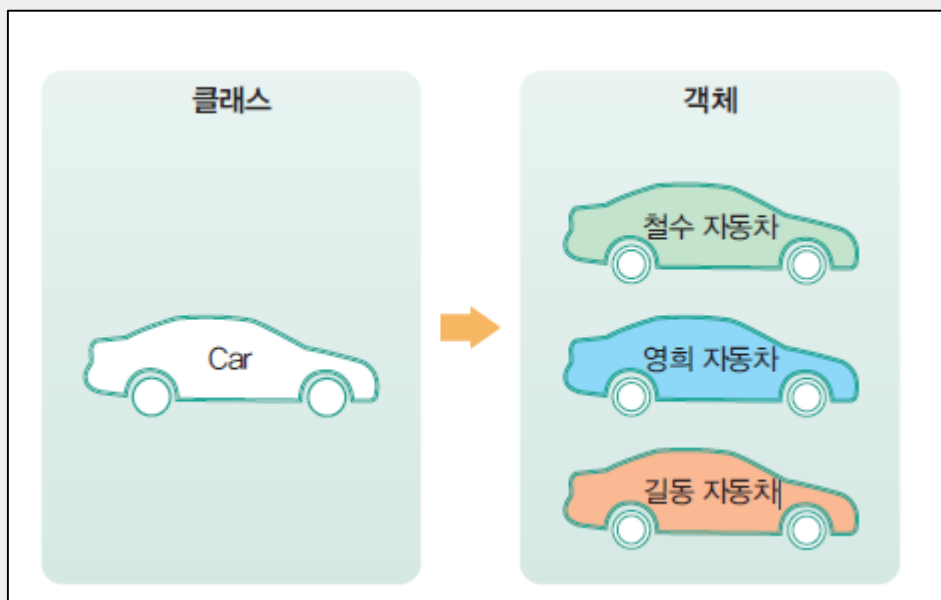
객체란?

- 객체(object)는 속성과 동작을 가진다.
- 예를 들어, 자동차는 메이커나 모델, 색상, 연식, 가격 같은 속성(attribute)을 가지고 있다. 또 자동차는 주행할 수 있고, 방향을 전환하거나 정지할 수 있다. 이러한 것을 객체의 동작(action), 메소드라고 한다.



클래스

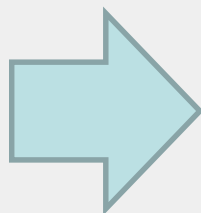
- 객체에 대한 설계도를 **클래스(class)**라고 한다. 클래스란 특정한 종류의 객체들을 찍어내는 형틀(template) 또는 청사진(blueprint)이라고도 할 수 있다.
- 클래스로부터 만들어지는 객체를 그 클래스의 **인스턴스(instance)**라고 한다.



예시

```
class Cat    #클래스 정의
    pass

nabi=Cat()   #인스턴스 생성
```



```
class Cat:

    def meow(self):
        print('야옹 야옹 ~~~')

nabi = Cat()
nabi.meow()
```

각각의 인스턴스들이 동일한 메소드를 공유

```
class Cat:

    def meow(self):
        print('야옹 야옹 ~~~')

nabi = Cat()
nabi.meow()

mimi = Cat()
mimi.meow()
```

클래스 작성하기

Syntax: 클래스 정의

형식 class 클래스이름 :

def __init__(self, ...) :

...

def 메소드1(self, ...) :

...

def 메소드2(self, ...) :

...

생성자(초기화 메소드)

: 인스턴스를 생성할 때 자동으로 실행되는 메소드

self는 식별자

예 class Counter:

def __init__(self):

self.count = 0

생성자를 정의한다.

def increment(self):

self.count += 1

메소드를 정의한다.

생성자(초기화 메소드)

- 생성자는 일반적으로 객체의 **인스턴스 변수**들을 정의하고 초기화한다.

```
class Counter:  
  
    def __init__(self) :  
        self.count = 0  
  
    def increment(self):  
        self.count += 1
```

객체의 멤버 접근

객체이름.변수

객체이름.메소드(인수)

```
class Counter:
    def __init__(self):
        self.count = 0
    def increment(self):
        self.count += 1
```

예.

```
a=Counter()
a.increment()
print(a.count)
```

```
class Calsum:

    def __init__(self, name) :

        self.count = 0

        self.name=name

    def hap(self, num1,num2):

        self.sum= num1+num2
```

```
#객체생성

a=Calsum("hong")

a.hap(3,5)

print(a.name, a.sum)
```

하나의 클래스로 객체를 많이 만들 수 있다.

```
class Counter:
```

```
    def __init__(self, initValue=0):
```

```
        self.count = initValue
```

```
    def increment(self):
```

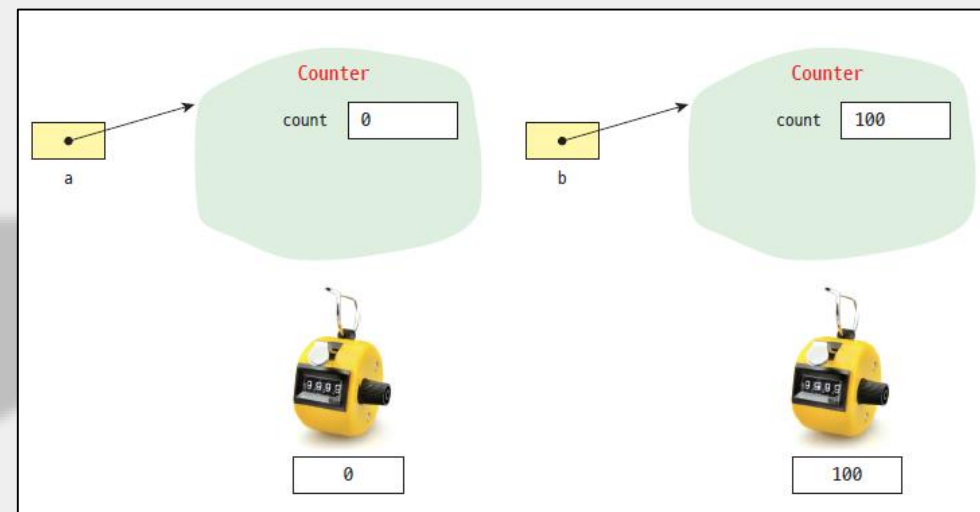
```
        self.count += 1
```

```
a = Counter(0)                # 계수기를 0으로 초기화.
```

```
b = Counter(100)             # 계수기를 100으로 초기화.
```

```
c = Counter( )
```

생성자가 매개 변수를 가지고 있으며
만약 사용자가 값을 전달하지 않으면
0으로 생각한다.



Human 클래스

```
class Human:
    def __init__(self):
        self.name="홍길동"
    def test(self):
        print("나는", self.name)
```

```
lee=Human()
kim=Human()
lee.test()
print(lee.name)
kim.test()
```

나는 홍길동
홍길동
나는 홍길동



```
class Human:
    def __init__(self, name):
        self.name=name
    def test(self):
        print("나는", self.name)
```

```
lee=Human("이순신")
kim=Human("이성계 ")
lee.test()
kim.test()
```

나는 이순신
나는 이성계

Cat 클래스

```
class Cat:
    def __init__(self, name, color):
        self.name = name
        self.color = color
    def meow(self):
        print(self.name, self.color)
nabi = Cat('나비', '검정색')
nero = Cat('네로', '흰색')
mimi = Cat('미미', '갈색')
nabi.meow()
nero.meow()
mimi.meow()
```

```
class Cat:
    def __init__(self, name, color):
        ....
#
#
nabi = Cat('나비', '검정색')
nero = Cat('네로', '흰색')
mimi = Cat('미미', '갈색')
```

nero 인스턴스 생성

mimi 인스턴스 생성

객체 생성



```
class Counter:
```

```
    def __init__(self):
```

```
        self.count = 0
```

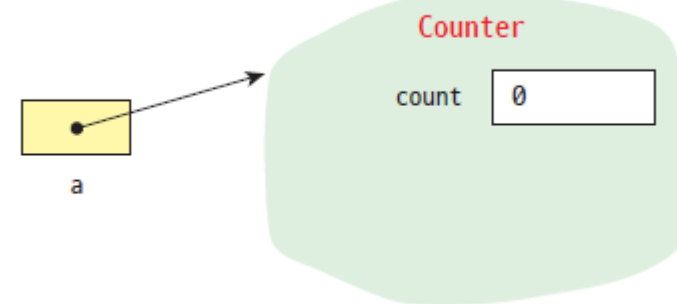
```
    def increment(self):
```

```
        self.count += 1
```

생성자 정의

인스턴스 변수 생성

```
a = Counter()
```



객체의 멤버 접근

```
class Counter:
    def __init__(self):
        self.count = 0
    def increment(self):
        self.count += 1

a = Counter()
a.increment()
print("카운터의 값=", a.count)
```

카운터의 값= 1

인스턴스 변수와 지역변수

- self를 붙인 인스턴스 변수의 범위는 클래스 전체
- self를 붙이지 않은 변수는 지역변수
- 지역변수는 그 함수(메소드) 안에 정의된 변수

```
def show(self):
```

```
    s= 100    #지역변수
```

```
    print(s, self.count)
```

중간점검

1. 클래스를 정의하는 데 사용되는 구문은 무엇인가?
2. 클래스 이름은 일반적으로 어떻게 작명하는가?
3. 클래스의 인스턴스는 어떻게 생성하는가?
4. 인스턴스의 속성과 메소드에 어떻게 접근하는가?
5. 메소드는 무엇인가?
6. Self의 목적은 무엇인가?
7. __init__() 메소드의 목적은 무엇인가?



실습: TV 클래스 정의

- TV 클래스를 작성해보자.

객체=변수+메소드



텔레비전 객체

변수

- channel
- volume
- on

메소드

```
show()  
setChannel()  
getChannel()
```

클래스 정의



```
class Television:
    def __init__(self, channel, volume, on):
        self.channel = channel
        self.volume = volume
        self.on = on

    def show(self):
        print(self.channel, self.volume, self.on)

    def setChannel(self, channel):
        self.channel = channel

    def getChannel(self):
        return self.channel
```

```
t = Television(9, 10, True)
```

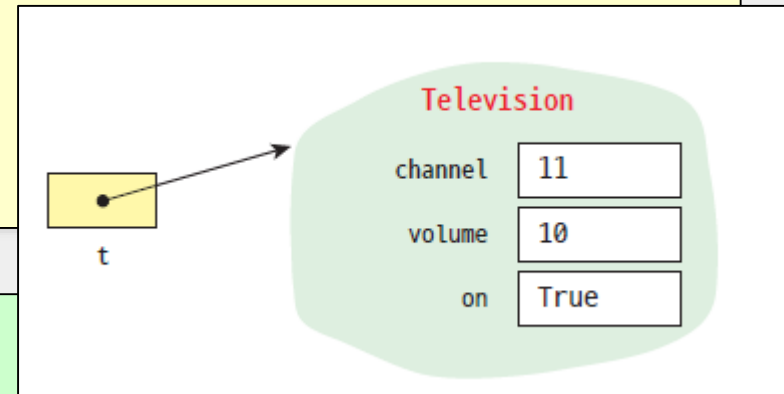
```
t.show()
```

```
t.setChannel(11)
```

```
t.show()
```

```
9 10 True
```

```
11 10 True
```



실습: 원 클래스 정의

- 원을 클래스로 표현해보자. 클래스 이름은 Circle로 하자. 원을 초기화하는 생성자에서 반지름을 속성으로 가진다. 메소드로는 원의 넓이와 둘레를 반환하는 `getArea()`와 `getPerimeter()`를 정의한다. `c = Circle(10)`

원의 면적 314.1592653589793

원의 둘레 62.83185307179586

Solution:

```
import math

# Circle 클래스를 정의한다.
class Circle:
    def __init__(self, radius = 0):
        self.radius = radius

    def getArea(self):
        return math.pi * self.radius * self.radius

    def getPerimeter(self):
        return 2 * math.pi * self.radius

# Circle 객체를 생성한다.
c = Circle(10)
print("원의 면적", c.getArea())
print("원의 면적", c.getPerimeter())
```

실습:자동차 클래스 정의

- 자동차는 메이커나 모델, 색상, 연식, 가격 같은 속성(attribute)을 가지고 있다. 또 자동차는 주행할 수 있고, 방향을 전환하거나 주차할 수 있다. 이러한 것을 객체의 동작(action)이라고 한다.
- Drive() 메소드를 실행하면 속도가 60으로 변경된다.

```
myCar = Car(0, "blue", "E-class")  
myCar.drive()
```

자동차 객체를 생성하였습니다.
자동차의 속도는 0
자동차의 색상은 blue
자동차의 모델은 E-class
자동차의 속도는 60

속성
메이커
모델
색상
연식
가격



동작
주행하기
방향바꾸기
주차하기

Solution:

```
class Car:
    def __init__(self, speed, color, model):
        self.speed = speed
        self.color = color
        self.model = model

    def drive(self):
        self.speed = 60

myCar = Car(0, "blue", "E-class")

print("자동차 객체를 생성하였습니다.")
print("자동차의 속도는", myCar.speed)
print("자동차의 색상은", myCar.color)
print("자동차의 모델은", myCar.model)

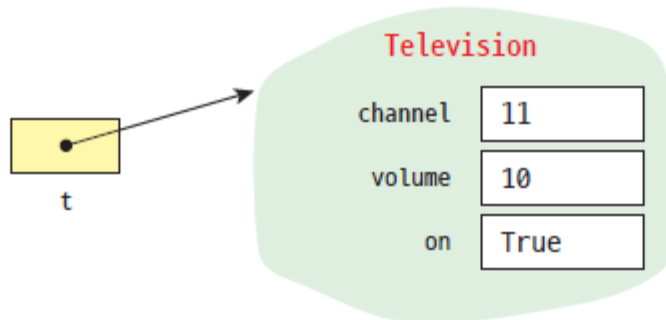
myCar.drive()
print("자동차의 속도는", myCar.speed)
```

객체 참조

- 파이썬에서 변수는 실제로 객체를 저장하지 않는다. 변수는 단지 객체의 메모리 주소를 저장한다. 객체 자체는 메모리의 다른 곳에 생성된다.

```
class Television:
    def __init__(self, channel, volume, on):
        self.channel = channel
        self.volume = volume
        self.on = on
    def setChannel(self, channel):
        self.channel = channel

t = Television(11, 10, True)
```

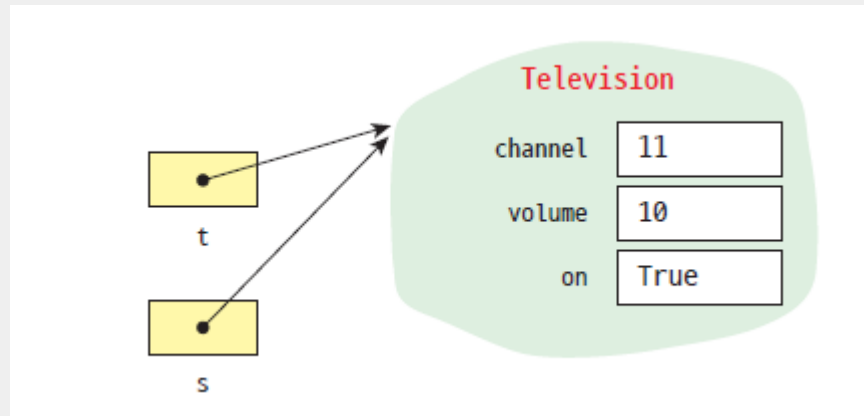


참조 공유



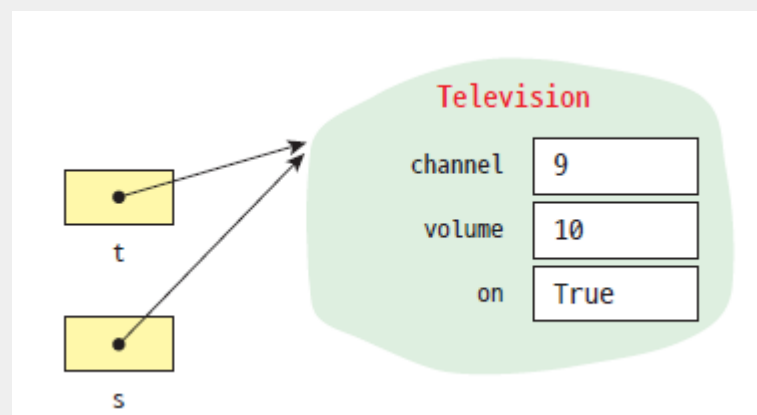
인하대학교

```
t = Television(11, 10, True)  
s = t
```



참조 공유

```
t = Television(11, 10, True)
s = t
s.channel = 9
```



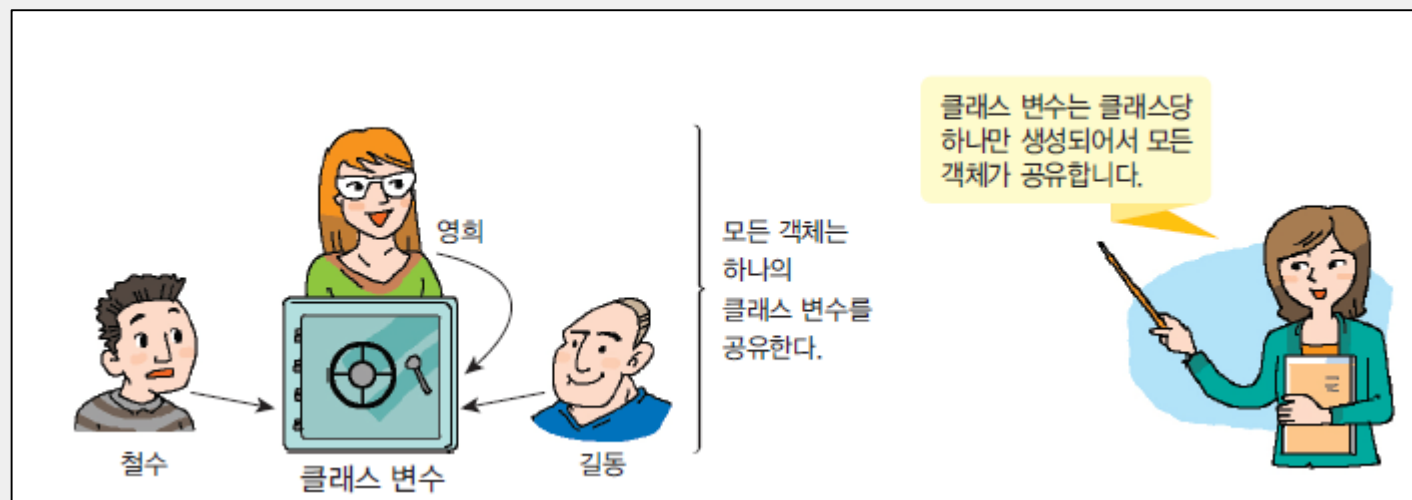
is, is not

- 2개의 변수가 동일한 객체를 참조하고 있는지를 검사하는 연산자가 있다.
- **is**와 **is not**

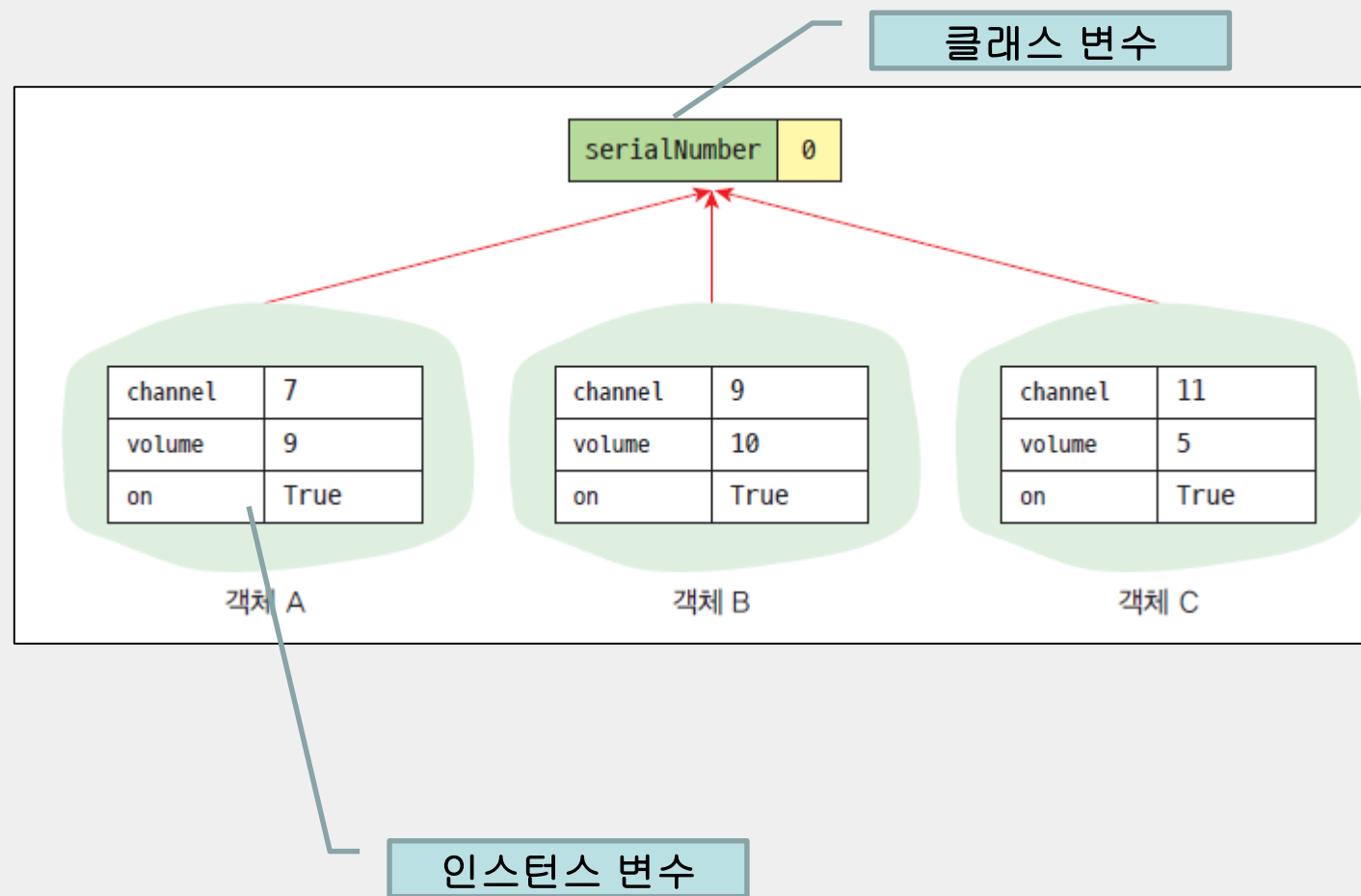
```
if s is t :  
    print("2개의 변수는 동일한 객체를 참조하고 있습니다.")  
  
if s is not t :  
    print("2개의 변수는 다른 객체를 참조하고 있습니다.")
```

클래스 변수

- 모든 객체를 통틀어서 하나만 생성되고 **모든 객체가** 이것을 공유하는 변수



인스턴스 변수 VS 클래스 변수



클래스 변수

```
class Television:
    serialNumber = 0                # 이것이 클래스 변수이다.

    def __init__(self, channel, volume, on):
        self.channel = channel
        self.volume = volume
        self.on = on
        Television.serialNumber += 1    # 클래스 변수를 하나 증가한다.
        self.number = Television.serialNumber

    def show(self):
        print(self.channel, self.volume, self.on, self.number)

myTV = Television(11, 10, True);
myTV.show()
```

11 10 True 1

인스턴스가 생성될때마다 1씩 증가

상수 정의

- 상수들은 흔히 클래스 변수로 정의된다.

```
class Monster :  
    # 상수 값 정의  
    WEAK = 0  
    NORMAL = 10  
    STRONG = 20  
    VERY STRONG = 30  
  
    def __init__(self) :  
        self._health = Monster.NORMAL  
  
    def eat(self) :  
        self._health = Monster.STRONG  
  
    def attack(self) :  
        self._health = Monster.WEAK
```

실습: 클래스 변수

- 어떤 섬에 강아지들이 있는데 강아지의 품종은 모두 같다고 하자. 그렇다면 강아지 객체마다 품종을 저장할 필요는 없을 것이다. 강아지의 품종은 클래스 변수로 정의하여도 된다.

```
class Dog:
    kind = "Bulldog"           # 클래스 변수
    def __init__(self, name, age):
        self.name = name       # 각 인스턴스에 유일한 인스턴스 변수
        self.age = age         # 각 인스턴스에 유일한 인스턴스 변수
        print(self.name, self, age, Dog.kind)
```

특수 메소드

- 파이썬에는 연산자(+, -, *, /)에 관련된 특수 메소드(special method)가 있다.
- 이들 메소드는 특수한 상황에 자동으로 호출. 객체에 대하여 +, -, *, /와 같은 연산을 적용하면 자동으로 호출된다.

```
class Circle:
    ...
    def __eq__(self, other):
        return self.radius == other.radius

c1 = Circle(10)
c2 = Circle(10)
if c1 == c2:
    print("원의 반지름은 동일합니다. ")
```

특수 메소드

연산자	메소드	설명
<code>x + y</code>	<code>__add__(self, y)</code>	덧셈
<code>x - y</code>	<code>__sub__(self, y)</code>	뺄셈
<code>x * y</code>	<code>__mul__(self, y)</code>	곱셈
<code>x / y</code>	<code>__truediv__(self, y)</code>	실수나눗셈
<code>x // y</code>	<code>__floordiv__(self, y)</code>	정수나눗셈
<code>x % y</code>	<code>__mod__(self, y)</code>	나머지
<code>divmod(x, y)</code>	<code>__divmod__(self, y)</code>	실수나눗셈과 나머지
<code>x ** y</code>	<code>__pow__(self, y)</code>	지수
<code>x << y</code>	<code>__lshift__(self, y)</code>	왼쪽 비트 이동
<code>x >> y</code>	<code>__rshift__(self, y)</code>	오른쪽 비트 이동
<code>x <= y</code>	<code>__le__(self, y)</code>	less than or equal(작거나 같다)
<code>x < y</code>	<code>__lt__(self, y)</code>	less than(작다)
<code>x >= y</code>	<code>__ge__(self, y)</code>	greater than or equal(크거나 같다)
<code>x > y</code>	<code>__gt__(self, y)</code>	greater than(크다)
<code>x == y</code>	<code>__eq__(self, y)</code>	같다
<code>x != y</code>	<code>__neq__(self, y)</code>	같지않다

특수 메소드 : __str__()

- 객체를 print()로 출력할 때 자동으로 호출되는 메소드.

```
class Counter:
    def __init__(self, initValue=0) :
        self.count = initValue

    def increment(self) :
        self.count += 1

    def __str__(self) :
        msg="카운트값"+str(self.count)
        return msg
```

```
a = Counter(100)
print(a)
```

실습: 주사위 클래스

- 주사위의 속성
 - 주사위의 값(value), 기본 1
 - 화면에 그려질 주사위의 크기(size), 30
 - 화면에 그려질 주사위의 위치(x, y)
- 주사위의 동작
 - 주사위를 생성하는 연산(__init__)
 - 주사위를 던지는 연산(roll_dice), 1~6 랜덤수
 - 주사위의 값을 읽는 연산(read_dice)
 - 주사위를 화면에 출력하는 연산(print_dice)



Solution:

```
from random import randint

class Dice :

    def __init__(self, x, y) :
        self._x = x
        self._y = y
        self._size = 30
        self._value = 1

    def read_dice(self) :
        return self._value

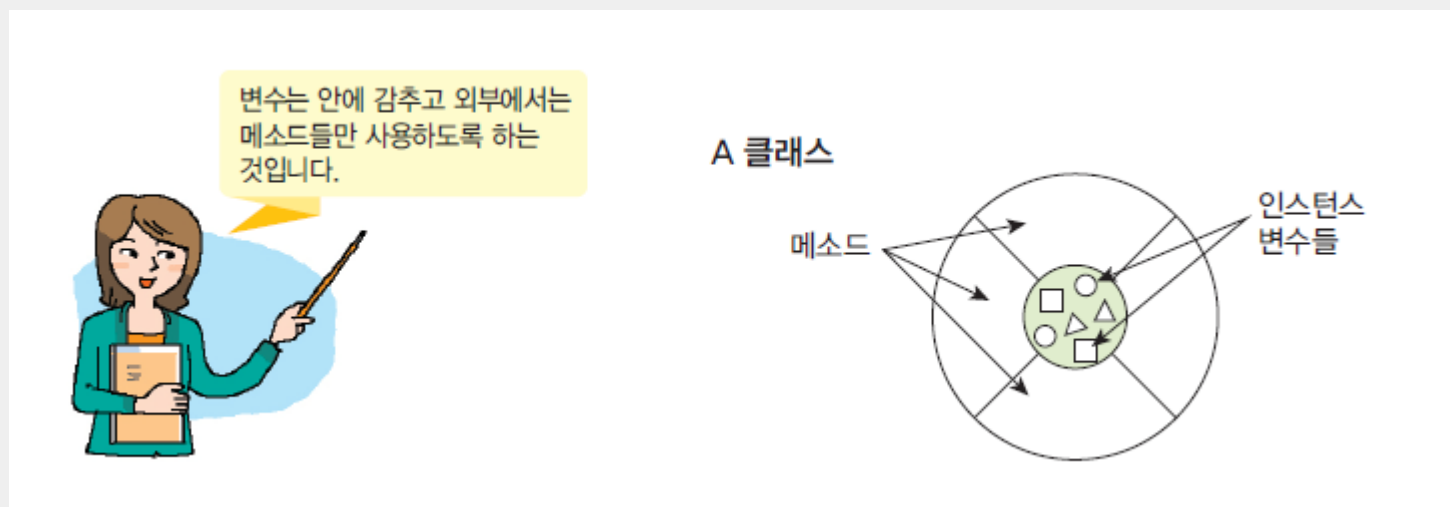
    def print_dice(self) :
        print("주사위의 값=", self._value)

    def roll_dice(self) :
        self._value = randint(1, 6)

d = Dice(100, 100)
d.roll_dice()
d.print_dice()
```

정보 은닉

- 파이썬에서 인스턴스 변수를 private으로 정의하려면 변수 이름 앞에 `_`을 붙이면 된다.
- `private`이 붙은 인스턴스 변수는 클래스 내부에서만 접근될 수 있다.




```
class Student:
    def __init__(self, name=None, age=0):
        self.__name = name        # __가 변수 앞에 붙으면 외부에서 변경 금지
        self.__age = age          # __가 변수 앞에 붙으면 외부에서 변경 금지

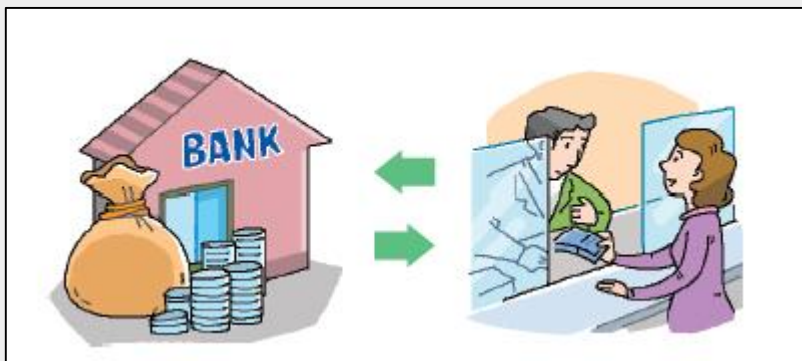
obj=Student()
print(obj.__age)
```

```
...
AttributeError: 'Student' object has no attribute '__age'
```

실습:은행 계좌

- 우리는 은행 계좌에 돈을 저금할 수 있고 인출할 수도 있다. 은행 계좌를 클래스로 모델링하여 보자. 은행 계좌는 현재 잔액(balance)만을 인스턴스 변수로 가진다. 생성자와 인출 메소드 `withdraw()`와 저축 메소드 `deposit()` 만을 가정하자. 은행 계좌의 잔액은 외부에서 직접 접근하지 못하도록 하라.

통장에서 100 가 출금되었음
통장에 10 가 입금되었음



Solution:

```
class BankAccount:
    def __init__(self):
        self.__balance = 0

    def withdraw(self, amount):
        self.__balance -= amount
        print("통장에서 ", amount, "가 출금되었음 ")
        print('잔액:', self.__balance)
        return self.__balance

    def deposit(self, amount):
        self.__balance += amount
        print(" 통장에 ", amount, " 가 입금되었음")
        print('잔액:', self.__balance)
        return self.__balance

a = BankAccount()
a.deposit(100)
a.withdraw(10)
```



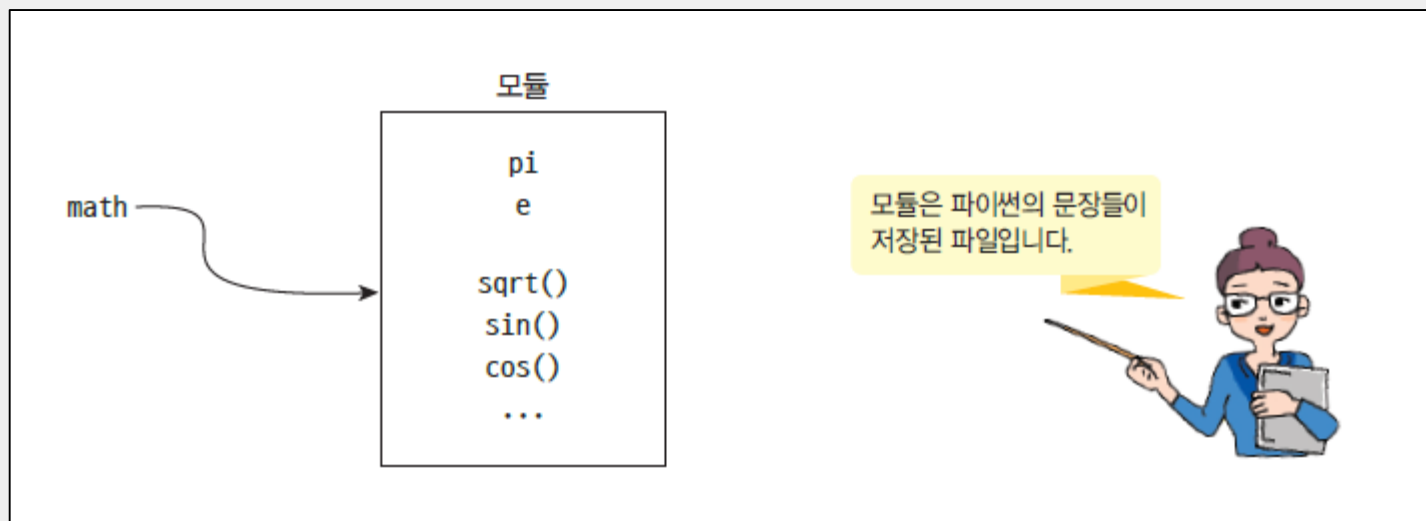
인하대학교

모범

모듈

- 파이썬에서 모듈(module)이란 함수나 변수 또는 클래스 등을 모아놓은 파일이다.
- 함수는 특정역할을 수행하는 **실행문들의 모음**에 이름을 붙인 것
- 필요한 모듈이 있으면 프로그램 개발시간 단축
- 파이썬은 수많은 모듈들을 기본적으로 제공: Built-in Module, 표준모듈

<https://docs.python.org/3/py-modindex.html>



- 모듈별로 제공되는 함수는 dir() 함수로 알 수 있음.
- 예, math 모듈이 제공하는 함수의 목록확인

```
import math
dir(math)
```

출력 결과

```
['__doc__', '__loader__', '__name__', '__package__', '__spec__', 'acos',
'acosh', 'asin', 'asinh', 'atan', 'atan2', 'atanh', 'ceil', 'copysign',
'cos', 'cosh', 'degrees', 'e', 'erf', 'erfc', 'exp', 'expm1', 'fabs',
'factorial', 'floor', 'fmod', 'frexp', 'fsum', 'gamma', 'gcd', 'hypot',
'inf', 'isclose', 'isfinite', 'isinf', 'isnan', 'ldexp', 'lgamma', 'log',
'log10', 'log1p', 'log2', 'modf', 'nan', 'pi', 'pow', 'radians', 'sin',
'sinh', 'sqrt', 'tan', 'tanh', 'trunc']
```

모듈 사용법

- 기본사용법

```
>>>import 모듈명  
ex) >>>import turtle, random
```

- 별칭사용

```
>>>import 모듈명 as 별칭  
ex) >>>import turtle as t
```

- 특정함수만 로딩

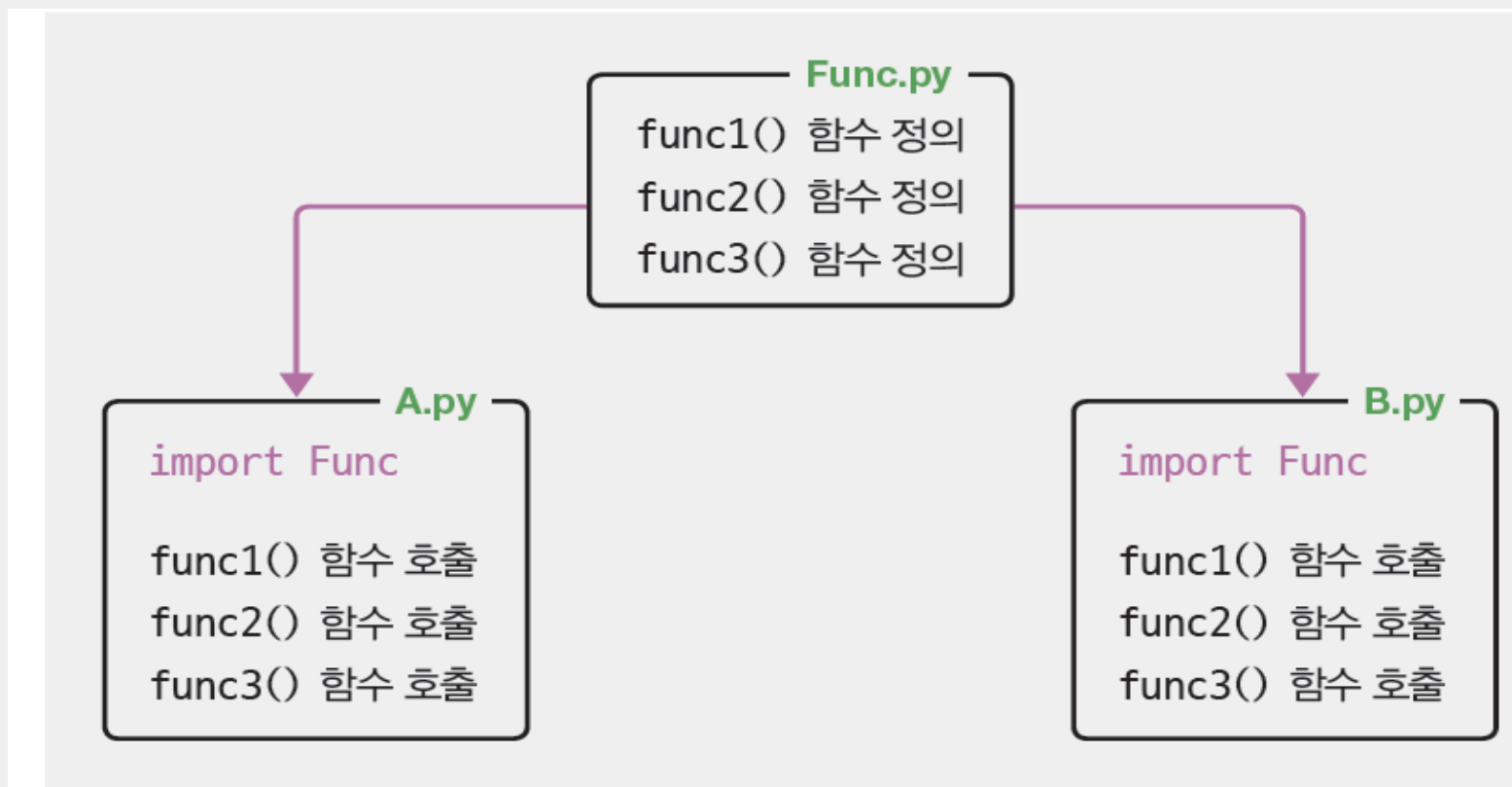
```
>>>from 모듈명 import 함수명  
ex) >>>from random import randint  
=> Randint 함수를 사용할 때 모듈명을 사용하지 않아도 됨
```

- 모든 함수 로딩

```
>>>from 모듈명 import *  
=> 함수를 사용할 때 모듈명을 사용하지 않아도 됨  
모듈내의 변수도 그대로 사용가능
```

사용자 정의 모듈

- 많이 사용하는 함수를 모아서 모듈로 만들어 놓고, 프로그램에서 해당 함수를 사용할 때 모듈을 Import하여 사용하면 편리함



사용자 정의 모듈

- 모듈은 .py 파일
- 연습을 위해 두개의 파일이 필요 : 모듈파일과 내가 만든 파일(둘다 확장자는 .py)
- 두 파일을 만들어서 같은 폴더에 저장.

Func.py

```
1  ## 함수 정의 부분
2  def func1() :
3      print("Func.py의 func1()이 호출됨.")
4
5  def func2() :
6      print("Func.py의 func2()가 호출됨.")
7
8  def func3() :
9      print("Func.py의 func3()이 호출됨.")
```

a.py

```
1  import Func
2
3  ## 메인 코드 부분
4  Func.func1()
5  Func.func2()
6  Func.func3()
```

파일명 : calc.py

```
def plus(a, b):  
    return a+b  
  
def minus(a, b):  
    return a-b  
  
def multiply(a, b):  
    return a*b;  
  
def divide(a, b):  
    return a/b
```

파일명 : test.py

```
import calc  
  
hap=calc.plus(10, 5)  
print(hap)  
  
print(calc.minus(10, 5))  
print(calc.multiply(10, 5))  
print(calc.divide(10, 5))
```

실습

import 모듈

```
import calc
```

```
print(calc.plus(10, 5))
```

```
print(calc.minus(10, 5))
```

```
print(calc.multiply(10, 5))
```

```
print(calc.divide(10, 5))
```

from 모듈 import 변수 또는 함수

```
from calc import *
```

```
print(plus(10, 5))
```

```
print(minus(10, 5))
```

```
print(multiply(10, 5))
```

```
print(divide(10, 5))
```



실습 : 삼각형, 사각형, 원의 면적 구하는 모듈

파일명 : area.py

```
def tri(x,y):  
    area3=x*y/2  
    return area3  
  
def rect(x,y):  
    area4=x*y  
    return area4  
  
def circle(x):  
    area0=3.14*x**2  
    return area0
```

파일명 : mod_test.py

```
import area  
  
a=5  
b=10  
  
result=area.rect(a,b)  
  
print("면적은 ", result)
```

파일의 위치



인하대학교

Case1



py_test

모듈파일 : area.py

내파일: mod_test.py

Case2



py_test

내파일: mod_test.py



my_modules

모듈파일 : area.py

모듈 탐색 경로

- 모듈을 호출하면 기본적으로 Built-in 내장 module을 먼저 탐색해서 찾음
- 만약 없다면, 파이썬은 `sys.path`에 정의되어 있는 디렉토리에서 모듈 파일을 탐색(import sys)
- 탐색순서
 1. 현재 작업 디렉토리
 2. PYTHONPATH 환경 변수에 등록된 디렉토리(sys.path)
 3. 파이썬이 설치된 경로 및 표준 라이브러리 디렉토리

예: C:\Python39\Lib (파이썬이 설치된 경로의 Lib 임)

경로추가



- 경로를 sys.path에 추가 : `sys.path.append('경로')`

ex) `>>>sys.path.append('d:\Wclass\Wmymodules')`

- 경로를 삭제 : `sys.path.remove('경로')`

①

```
>>> import sys
```

②

```
>>> sys.path
```

```
['', 'C:\\Users\\inha\\AppData\\Local\\Programs\\Python\\Python39\\Lib\\site-packages', 'D:\\파이썬작업\\오리', 'C:\\Users\\inha\\AppData\\Local\\Programs\\Python\\Python39\\python39.zip', 'C:\\Users\\inha\\AppData\\Local\\Programs\\Python\\Python39\\DLLs', 'C:\\Users\\inha\\AppData\\Local\\Programs\\Python\\Python39\\Scripts', 'C:\\Users\\inha\\AppData\\Local\\Programs\\Python\\Python39', 'C:\\Users\\inha\\AppData\\Local\\Programs\\Python\\Python39\\lib\\site-packages']
```

③

```
>>> sys.path.append('d:\\py_test')
```

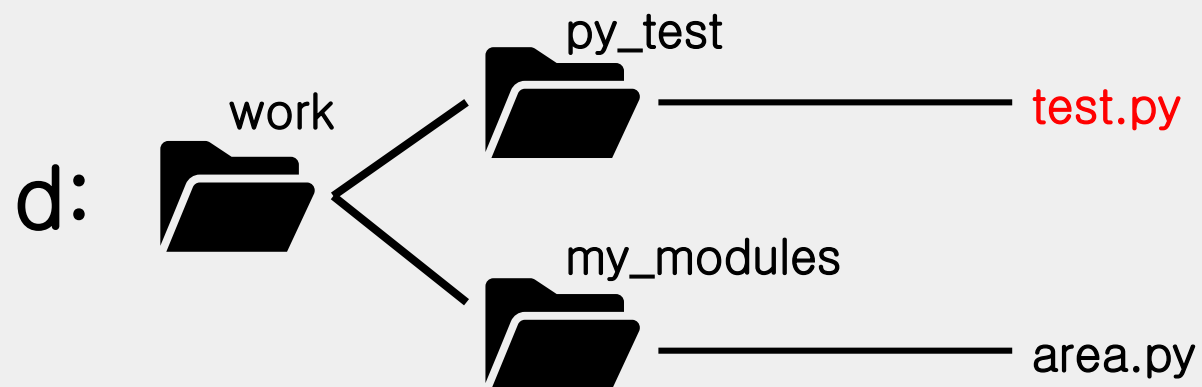


파일입출력

파일의 기초

- 프로그램에서 만든 데이터를 영구히 저장하고자 한다면 하드 디스크에 파일 형태로 저장하여야 한다.

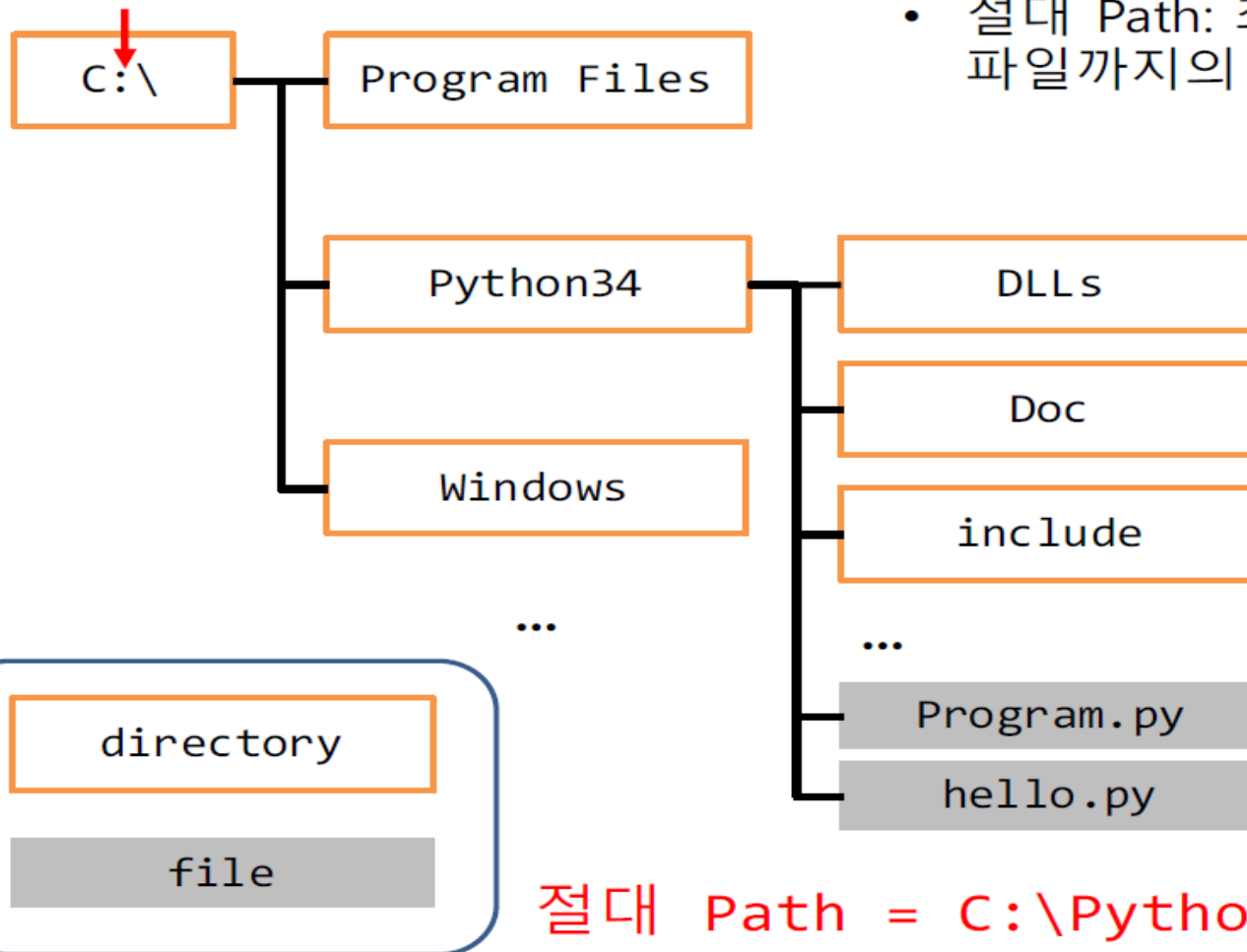




1. 상대경로
2. 절대경로

파일위치 : 절대경로 vs. 상대경로

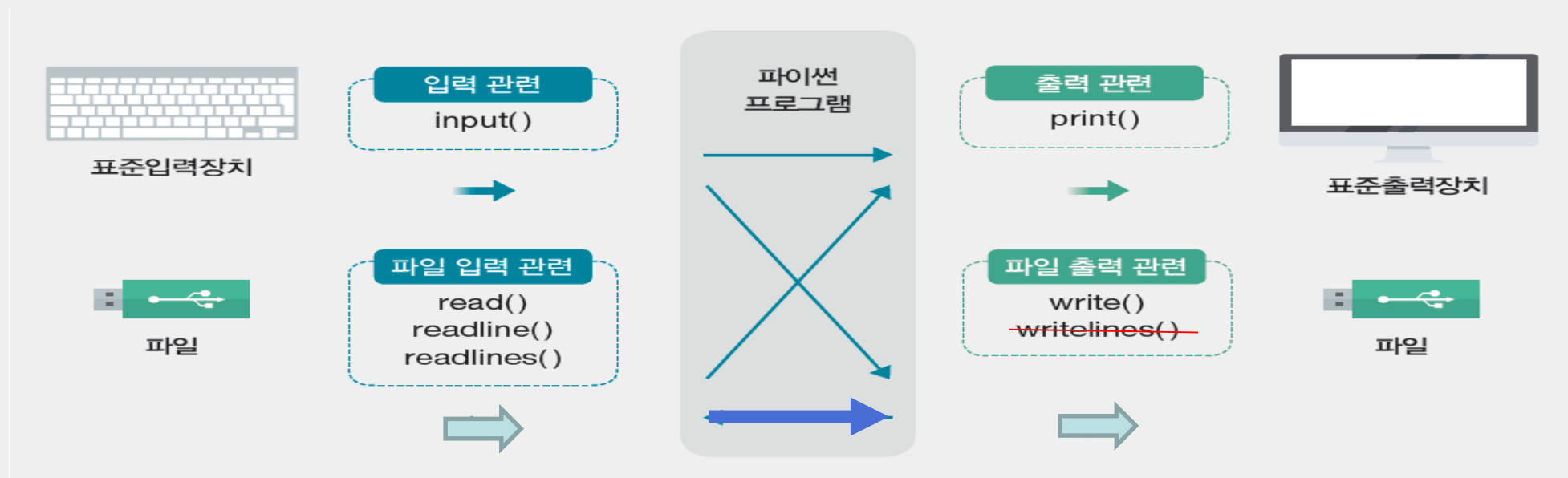
최상위 Directory



- 절대 Path: 최상위 디렉토리부터 파일까지의 모든 디렉토리를 나열

절대 Path = C:\Python34\hello.py

파일입출력

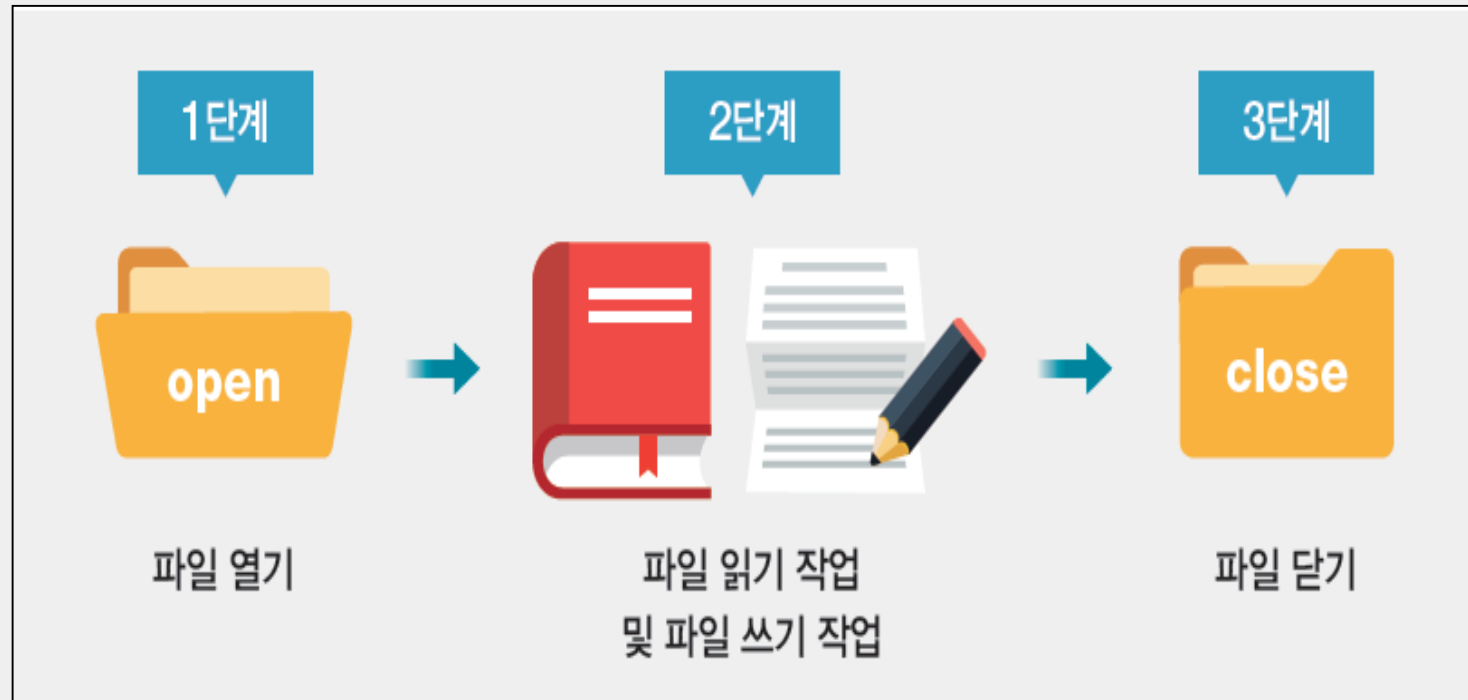


- ❖ 키보드로 입력하는 것을 표준 입력, 모니터로 출력하는 것을 표준 출력이라 함.
- ❖ `input( )` 함수와 `print( )` 함수는 키보드 및 모니터의 입출력
- ❖ `read( )`, `readline( )`, `readlines( )`는 파일 내용을 읽음
- ❖ `write( )`는 파일에 내용을 씀

파일입출력



인하대학교



파일 열고 닫기

Syntax: 함수 정의

형식 파일객체 = `open`(파일이름, 파일모드)
파일객체.close()

예 `infile = open("input.txt", "r")`
...
`infile.close()`

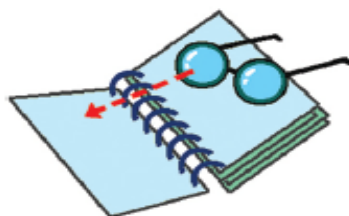
파일 객체

파일의 이름(name)

파일을 여는 모드(mode)

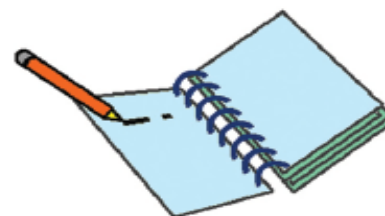
파일 모드

파일 모드	모드 이름	설명
"r"	읽기 모드(read mode)	파일의 처음부터 읽다.
"w"	쓰기 모드(write mode)	파일의 처음부터 쓴다. 파일이 없으면 생성된다. 만약 파일이 존재하면 기존의 내용은 지워진다.
"a"	추가 모드(append mode)	파일의 끝에 쓴다. 파일이 없으면 생성된다.
"r+"	읽기와 쓰기 모드	파일에 읽고 쓸 수 있는 모드이다. 모드를 변경하려면 seek()가 호출되어야 한다.



"r"

파일의 처음 부터 읽는다.



"w"

파일의 처음 부터 쓴다.
만약 파일이 존재하면 기존의
내용이 지워진다.



"a"

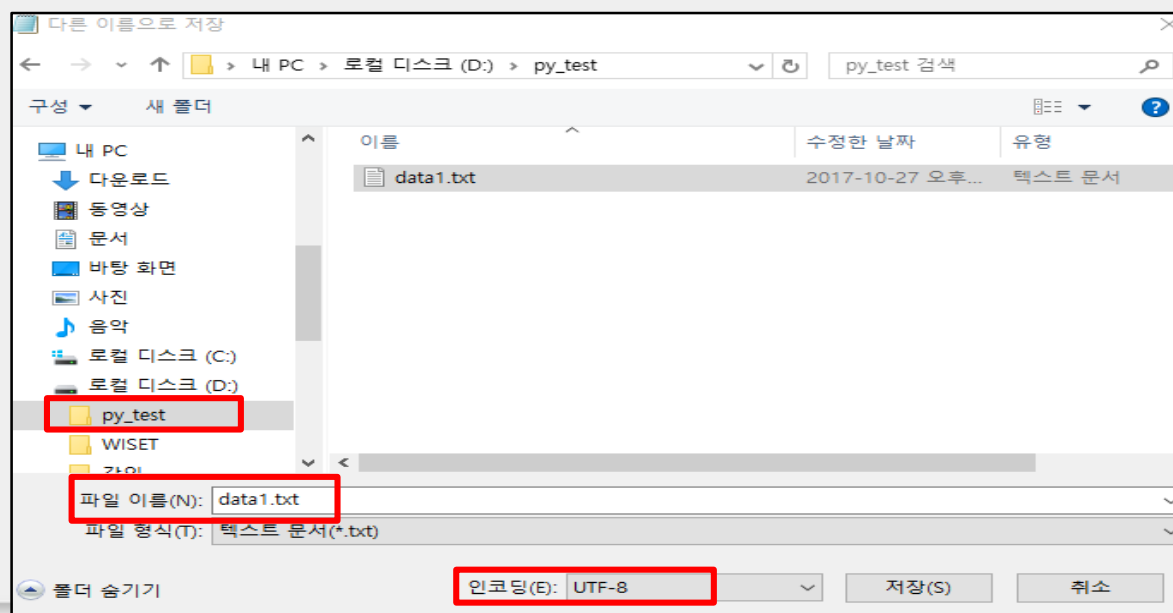
파일의 끝에 쓴다.
파일이 없으면 생성 된다.

파일에서 읽기

- 메모장에 다음과 같이 적어 **d:\py_test\data1.txt**로 저장

```
홍길동 문화경영학과 010-1234-5678  
김철수 연극영화학과 010-1234-5679  
김영희 통계학과 010-1234-5680
```

- 저장 창에서 인코딩을 **UTF-8**로 선택해야 파이썬에서 한글이 깨지지 않음



파일에서 읽기

```
infile = open("d:\py_test\data1.txt", "r", encoding='UTF8')  
lines = infile.read()  
print(lines)  
infile.close()
```

인코딩은 문자를 처리하는 방식

홍길동 문화경영학과 010-1234-5678
김철수 연극영화학과 010-1234-5679
김영희 통계학과 010-1234-5680

문자 인코딩

- 문자셋의 규칙대로 바이트를 만드는 방법을 인코딩(encoding)
- 최근에는 세계의 모든 문자를 나타낼 수 있는 유니코드가 사용된다.
- 유니코드 중에서 가장 많이 사용되는 인코딩은 UTF-8이다. UTF-8에서는 각 문자를 1개에서 4개의 바이트로 인코딩한다.

```
infile = open("input.txt", "r", encoding="utf-8")
```

한글자씩 읽기

```
infile = open("input.txt", "r")
ch = infile.read(1)
while ch != "" :
    print(ch)
    ch = infile.read(1)
infile.close()
```

...

파일에서 읽기

```
f = open("d:\\py_test\\data1.txt", "r", encoding='UTF8')
while True:
    content=f.readline()
    if content=="":
        break
    print(content, end="")
f.close()
```

홍길동 문화경영학과 010-1234-5678
김철수 연극영화학과 010-1234-5679
김영희 통계학과 010-1234-5680

1:홍길동 문화경영학과 010-1234-5678
2:김철수 연극영화학과 010-1234-5679
3:김영희 통계학과 010-1234-5680



파일에서 읽기

```
f = open("d:\py_test\data1.txt", "r", encoding='UTF-8')
contents=f.readlines()
for i in contents:
    print(i, end="")
f.close()
```

홍길동 문화경영학과 010-1234-5678

김철수 연극영화학과 010-1234-5679

김영희 통계학과 010-1234-5680

파일에서 읽기

```
f = open("d:\py_test\data1.txt", "r", encoding='UTF-8')
contents=f.readlines()
for i in contents:
    print(i, end="")
f.close()
```

```
with open("d:\py_test\data1.txt", "r", encoding='UTF-8') as f:
    contents=f.readlines()
    for i in contents:
        print(i, end="")
```

close() 할 필요없음

실습

```
fName=input("파일명을 입력하세요 : ")
```

```
f = open(fName, "r", encoding='UTF-8')
```

```
contents=f.readlines()
```

```
for i in contents:
```

```
    print(i,end="")
```

```
f.close()
```

파일명을 입력하세요 : d:\py_test\data1.txt

홍길동 문화경영학과 010-1234-5678

김철수 연극영화학과 010-1234-5679

김영희 통계학과 010-1234-5680



파일에 쓰기

```
outfile = open("d:Wpy_testWoutput.txt", "w")  
outfile.write("김영희Wn")  
outfile.close()
```

output.txt

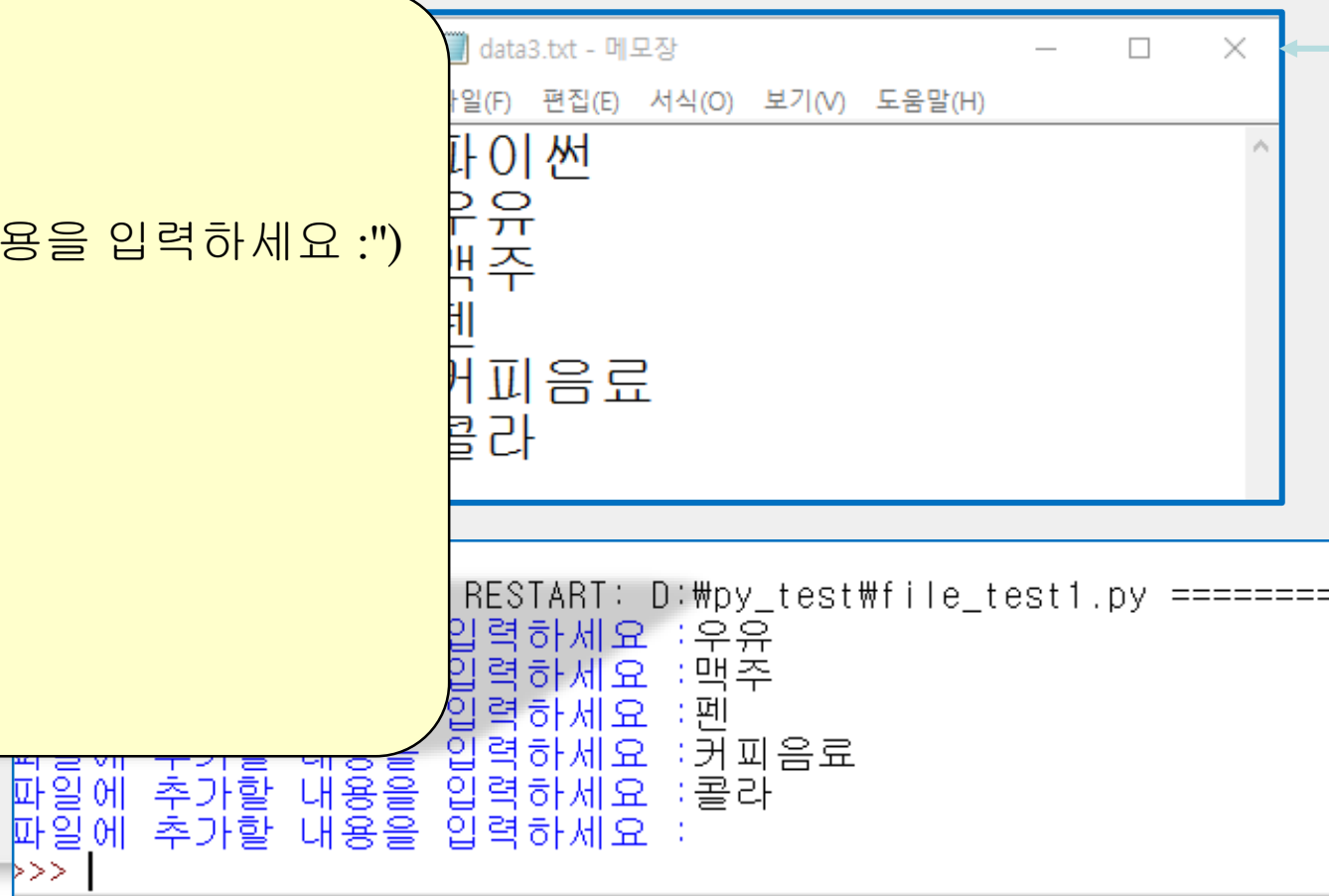
김영희

실습

- input() 함수를 통해 입력되는 값을 파일에 쓰기
- input() 함수 반복해서 사용
- 아무것도 입력하지 않고 Enter 를 누르면 파일에 쓰는 것을 종료하는 프로그램

```
out = open("d:\py_test\data3.txt","a")
while True:
    contents=input("파일에 추가할 내용을 입력하세요 :")
    if contents=="":
        break
    else:
        out.write(contents+"\n")
out.close()
```

data3.txt



```
data3.txt - 메모장
파일(F) 편집(E) 서식(O) 보기(V) 도움말(H)
파이썬
우유
맥주
커피음료
콜라

RESTART: D:\py_test\file_test1.py =====
입력하세요 : 우유
입력하세요 : 맥주
입력하세요 : 펜
입력하세요 : 커피음료
입력하세요 : 콜라
입력하세요 :
>>> |
```